

VayuNetra — User Guide & Project Reference

The single source of truth for what VayuNetra is, how to use every screen, and what every option does. If you have a doubt about any feature during prep, demo, or judging — the answer should be here. If it isn't, it's a bug in this document.

- Live app: <https://vayunetra-aqi.vercel.app> · API: <https://vayunetra-c8i8.onrender.com> · Telegram: [@aqivayu_bot](#)
- Team: **Omkar Kadam · Sejal Kumbhar · Abhinav Prasad** — Full-Stack AI Engineers (models · agents & API · app & channels)
- Built for ET AI Hackathon 2026, Problem Statement 5: *AI-Powered Urban Air Quality Intelligence for Smart City Intervention*.

1. What VayuNetra is, in one minute

India measures its air (900+ CAAQMS stations) and forecasts it (SAFAR), but almost no city can turn a bad reading into a *specific, attributed, delivered intervention*. VayuNetra is the **operate** layer that closes that loop:

1. **Trace** — for every ~1 km² hexagon of the city, a machine-learning model says *who is to blame* (traffic, construction dust, industrial, biomass burning, transported, other), with SHAP evidence, and **abstains** to cited chemical-signature priors when it lacks out-of-sample skill.
2. **Predict** — a 24/48/72-hour PM2.5 forecast per cell with calibrated 80% uncertainty bands.
3. **Act** — a ranked enforcement worklist for officers: each item carries an evidence dossier (Sentinel-2 satellite patch + regulatory citations retrieved by RAG) and a one-click **draft Notice PDF** with a projected-impact chart. Officers approve/dispatch; the system then measures the before/after effect.
4. **Protect** — citizen health advisories in **eight languages** (Hindi, Kannada, Marathi, Tamil, Telugu, Bengali, Gujarati, English), delivered over the web app, a live Telegram bot, a working IVR line, and public-display boards — targeted using 5,495 vulnerability-scored zones (hospitals, schools, elder-care, outdoor-work sites × real population).

It runs **live in 10 Indian cities** — Delhi, Bengaluru, Mumbai, Hyderabad, Chennai, Kolkata, Pune, Ahmedabad, Jaipur, Lucknow — on **₹0 infrastructure** (Vercel + Render + Supabase + GitHub Actions free tiers). Signal-to-cited-recommendation is measured live at **0.8–9.7 seconds** of compute through a six-agent LangGraph pipeline; approval, dispatch and closure of each action are timestamped separately.

The two seams. Everything decouples through (1) the Supabase schema — models *write* rows, the API *reads* rows — and (2) the FastAPI contract — one `{success, data, error, meta}` envelope. That's why three people could build in parallel for weeks without blocking each other.

2. Getting in

I want to...	Do this
Just see it	Open vayunetra-aqi.vercel.app → click Open console
Run everything locally	<code>make install</code> once, then <code>make dev</code> (starts FastAPI :8000 + Vite :5173 in one terminal)

I want to...	Do this
Demo-proof mode (no internet/DB)	Set <code>DEMO_MODE=true</code> in <code>.env</code> (default) — every endpoint serves bundled fixtures; the whole flow works offline
Pre-demo smoke check	<code>make prewarm</code> ~15 min before a demo → prints GO / NO-GO after warming Render/Supabase and hitting every demo endpoint
Subscribe to citizen alerts	Telegram: message /start to @aqivayu_bot , pick a city

URLs: `/` is the landing page, `/console` is the operations console. (Old `#/console` links auto-upgrade to the clean path.)

Deep links. Every console state is a URL: `/console?city=hyderabad§ion=action&cell=8860a...&mode=satellite&layers=plumes,fires`. Switching city, section, cell, map mode or layers writes back to the address bar, so any view can be bookmarked or handed to someone. **Keyboard:** `1 – 7` jump sections, `[ / ]` cycle cities, `P` toggles presentation mode.

3. The Landing page (`/`)

The public front door — no login, loads fast (the heavy map code only loads at `/console`).

- **Top nav:** "How it works", "Architecture", "Platform", "Validation" (scroll links) · GitHub link · **Open console** button.
- **Live status line:** real Delhi worst-cell AQI and the last pipeline latency, fetched from the live API on load.
- **How it works:** the four steps (01 Trace · 02 Predict · 03 Act · 04 Protect).
- **Architecture:** the full system diagram (same `architecture.svg` used in the pitch deck — one source of truth).
- **Platform:** 12 feature cards.
- **The data at a glance:** live from `GET /landing/snapshot` (10-minute cache, stamped *as of HH:MM*) — Delhi source-mix donut, per-city PM2.5 now-vs-+24h, and the scale row (modelled cells · sources · vulnerability zones · live enforcement recommendations); a bundled snapshot is shown only until the API answers.
- **Validation table:** claim / result / how it was checked (cosine 0.88/0.90/0.93 vs published apportionment anchors; 2.30× rush-hour traffic SHAP; forecast skill vs persistence; CQR 75–80% coverage; the rejected TFT model; test counts).
- **QR codes:** open the app on your phone · subscribe on Telegram.
- **Footer:** product links, resources, team names.

4. The Console (`/console`) — the shell

4.1 Top bar

Control	What it does
Logo + "VayuNetra"	Back to the landing page
City selector (dropdown)	Switch between any of the 10 cities (Delhi · Bengaluru · Mumbai · Hyderabad · Chennai · Kolkata · Pune · Ahmedabad · Jaipur · Lucknow). <i>Everything</i> — map, panels, forecasts, worklists — follows the selected city. Your choice is remembered (localStorage).

Control	What it does
Section name	Where you are (the full explanation lives in the section's spine — see 4.5)
Present (or the P key)	Presentation mode — type scales up ~18% for a projector; layers card collapses to its chip. Persists across reload.
? (Help)	Replays the guided tour
GitHub icon	Repo link
← Landing	Back to /

If the backend is asleep (Render free tier cold start), an amber banner appears: *"backend waking up — showing bundled demo snapshot"* with **retry** and dismiss. It clears itself the moment any API call succeeds.

4.2 Guided tour

First visit shows a 5-step spotlight tour: ① the city switcher ② the map ("every hexagon is ~1 km² — click one") ③ the section spine (verb, one sentence, numbered path) ④ the Enforcement panel ⑤ the sidebar groups and keyboard shortcuts. **Esc**, backdrop click, or **Skip** ends it; it never auto-shows again (replay via the ? button).

4.3 Status strip (top-left of the map)

- **AQI card:** worst-cell AQI + category + PM2.5 + data freshness; pulsing green dot = live WebSocket connected.
- **Compound-risk chips** (only when conditions are real):
- **Heat×Smog WATCH/ALERT** — IMD heatwave criteria × CPCB bands (heat amplifies PM mortality).
- **GRAP Stage I–IV · from forecast** — our 24h forecast enters a statutory CAQM GRAP band *a day before* observed AQI would trigger it.
- **Dust×Traffic · N cells** — cells where construction dust AND traffic are both ≥25% of blame (traffic resuspends dust; these corridors escalate fastest).
- **Last pipeline run card:** wall-clock of the last full multi-agent run (e.g. **0.9s**), with the "signal→action < 5 min ✓" badge.

4.4 Navigation — 7 sections in 3 groups

Desktop: navy left sidebar, grouped by the question each section answers; the number beside each is its keyboard shortcut. Below 1280 px the rail collapses to icons (hover for the name). Mobile: the same 7 sections as a bottom tab bar; the map is a 42vh block and panels stack below it.

Group	Section	Verb	One-sentence purpose (shown in the spine)
Operate	Enforcement (1)	Act	Where to send an inspector today, with the evidence to back it — from the worst places on the map to a signed notice and a tracked outcome
	Forecast (2)	Anticipate	What the air will do in the next 72 hours, how much to trust that, who it will affect — and what happened before
	Advisories (3)	Inform	Tell residents what to do, in their language, on the channel they use — and let them report smoke back
Understand	Cities (4)	Compare	Ten cities on one scoreboard: who is worse, what drives it, which playbook worked elsewhere

Group	Section	Verb	One-sentence purpose (shown in the spine)
	Impact (6)	Fund	The funding case: annual burden, what NCAP would avert, where the money should go, whether the system is fair
Explore	Simulator (5)	Decide	Run an intervention on the model before spending money
	Pipeline (7)	Trust	The six AI agents, run live, every step traced

Keys: 1–7 sections · [] previous/next city · P presentation mode.

4.5 The section spine — how anyone knows what to do

Every section's right-hand rail starts with a sticky card: the **verb** (Act, Anticipate, Inform...), the section name, **one honest sentence** on what it is for, and a row of **numbered steps** — the path an officer takes through it (Enforcement: 1 Worst places now → 2 Ranked worklist → 3 Evidence & notice → 4 Dispatch by ward → 5 Track outcomes). Click a number to jump; the cards below carry the **same numbers** in a blue badge, and the strip highlights the step in view as you scroll. Every numbered card has a **?** — a plain-language popover saying *what* the card shows, *where the numbers come from* and *what to do with it*. This is the same information the speaker notes carry, so a judge exploring alone reads the same story a presenter tells.

5. The map

5.1 Three view modes (the Layers chip, top-right of the map — click to expand)

Mode	What the hexagons show
Sources (default)	Who is to blame — each cell filled by its dominant source's colour (traffic red · construction dust yellow · industrial purple · biomass burning orange · transported blue · other gray), opacity by dominance. Hover = top-3 shares + SHAP drivers.
Sat NO2	Sentinel-5P tropospheric NO ₂ column (blue low → red high) — independent satellite evidence.
PM2.5	The dense 1-km model field. Sub-toggle "Stations only" ↔ "Dense 1km" shows what the E2 downscaler adds beyond the sparse station grid; legend shows CPCB AQI bands and the skill note.

Play the last 24 hours — the ► control at the bottom of the map replays the trailing day: the dense PM2.5 field recolours hour by hour so the city visibly breathes through the evening peak and morning lull. Mechanics are labelled honestly ("station-scaled replay": each 1 km cell scaled by its nearest monitor's hourly ratio); it returns to live at the end.

5.2 Five overlay toggles (independent, all off by default)

- **Detected sources** — dots for the 647 registered (OSM) + satellite-detected (E1 CV) emission sources. **Hover a dot to see its real Sentinel-2 image** (worklist sources carry one; others show no image, never a stand-in).
- **Fire / burn events (30d)** — NASA FIRMS thermal detections over the city (the stubble/waste-burning layer), refreshed by `scripts/fetch_fire_events.py`.
- **Wind plumes** — Gaussian plume footprints from the top sources, oriented by the live wind field, with a **wind-arrow grid** across the city (direction; length = speed) so the plume orientation has a visible cause.
- **Ward boundaries** — administrative wards (Datameet / OSM, ODbL), **each filled by its mean PM2.5** (dense-field cells inside it, CPCB colours; the value shows on hover). These same polygons power the place names.

- **Freight corridors** — violet truck-route lines (OSM motorway/trunk) with entry-window policy notes (e.g. Delhi 23:00–07:00).

5.3 Cell Story (click any hexagon)

The heart of the console. Opens as a left drawer (desktop) / inline card (mobile):

1. **Header** — the *place name* ("R. K. Puram", "Ward K/E", "near Kalena Agrahara"), with the raw H3 cell id as a small subtitle. (An H3 id like 883da11215ffff is the globally unique address of that ~1 km² hexagon — Uber's H3 grid, resolution 8, is the spatial unit of the whole system.)
2. **"1 • Who's to blame"** — source-share bars + confidence. Below, one of two "why" boxes: - **Green (model attribution)**: SHAP drivers in µg/m³ and the model's own out-of-sample R² — "*passed the ≥0.15 skill gate*". - **Amber (chemical signature)**: when the local model lacks skill, the system says so and falls back to cited marker-chemistry priors — *it never over-claims*.
3. **"This place — past air"** — daily PM2.5 for this cell over 30 d / 90 d / 1 y drawn over the CPCB colour bands, a one-line verdict ("↑ Worse than a month ago (+20%) · mostly satisfactory over the last 30 days"), and red **spike-day markers** each with a data-backed reason (fire detections that day, weekday, or the excess over the two-week norm). Cells without a long station record honestly show the nearest monitored cell and say how far away it is.
4. **"2 • Where it's heading"** — the cell's +24/48/72h PM2.5 with 80% intervals, colour-coded by AQI band.
5. **"3 • Act — view enforcement actions →"** — jumps to Enforcement, sorted nearest-to-this-cell first.
6. **Share (↗ icon in the header)** — renders a WhatsApp-ready PNG card of this place (blame, verdict, 90-day trend, forecast); uses the phone share sheet where available, else downloads.

6. The seven sections, in depth

6.1 Enforcement (the officer loop)

- **Morning brief (step 1)**: the page a commissioner reads — city mean now vs the same hours yesterday (with its age if the last ingest is old), the worst place, the 24 h outlook, **where the air is about to turn** (cells whose calibrated P(Very Poor) ≥ 30 % at any horizon and that are not already Very Poor), the **top 3 actions** with place, source, contribution and exposure, **yesterday's dispatches with their measured effect**, and the advisory summary. **PDF** downloads it (same renderer as the notice); **Send to Telegram** pushes it to the city's subscribers; the daily pipeline sends it automatically when a bot token is configured. Every line is a template over stored rows — no language model.
- **Worklist (step 2)**: up to 10 ranked recommendations, each titled by **source type • place** ("Construction dust • Punjabi Bagh") with its priority score and evidence rubric as meta. Filters: **All / Construction / Industrial / Waste** chips + free-text search. If a cell is focused, sorting switches to *nearest first* (badge: "nearest to selected cell first"; cards gain a "~N km" or "📍 this cell" chip). Multiple identical detections in one cell collapse into one card with **"N similar sites here"** — one inspection covers the cluster.
- **Each card**: priority score, officer-rubric score (.../10), rationale (what the source is, its % contribution, residents exposed, what to inspect, regulatory basis).
- **"Evidence dossier"** expands in place: the **Sentinel-2 satellite patch** of the site (with detection confidence) and up to 3 deduplicated **regulatory citations** retrieved by RAG from the NCAP/GRAP/CPCB/Air-Act corpus, each with a match score.
- **"Notice PDF"** (step 3) downloads a draft enforcement notice: addressee block, cited provisions, 24-hour IST deadline, the satellite image, a **"Projected impact of compliance"** bar chart (forecast PM2.5 with vs without this source's share, at +24/48/72h — labelled a *screening estimate*), an authorisation block naming the issuing authority, and a **PROVENANCE** section (every figure read from structured model output by deterministic code; regulatory

text retrieved verbatim; no number from a language model). It is stamped "**DRAFT — pending officer authorisation**"; the system never auto-sends.

- **Status flow (buttons on the card):** **Approve / Dismiss** on a proposed item, **Dispatch team** once approved; the card then shows *Dispatched · tracking armed*. **Dispatching arms intervention tracking** — the cell's 7-day PM2.5 baseline is frozen and a before/after window opens. **Close case** on a dispatched item records the field finding (*violation found · compliant · site inaccessible · not applicable*) and an optional note; the card then shows *Closed · finding · date*.
- **Acting as** (input beside the search box): the officer's name stamped on every approve / dispatch / close. The demo runs without sign-in, so the trail records what the console was told; a real deployment binds this to the authenticated user.
- **History** (on any acted-upon card): the immutable audit trail — from → to, who, when, finding, note (`enforcement_status_log` , `GET /enforcement/{id}/log`).
- **Overnight:** the nightly enforcement run refreshes evidence and priority *in place* for anything an officer has acted on (same id, same status, same trail) and replaces only still-proposed items — approvals and dispatches never vanish with the next run.
- **Intervention tracking card (step 5):** shows each dispatched rec's measured effect as *cell delta minus city-wide drift* (marked "provisional" for the first 7 days) — honest impact measurement, not a victory lap.
- **City Intel card:** registry-source count, traffic index, sensitive-zone count, hospitals/schools line, top registered sources.
- **Ward dispatch queues (step 4):** approved and dispatched recommendations auto-group into per-ward field queues (the grid-supervision pattern) — wards resolved offline from the shipped boundary polygons. Before anything is dispatched the card says so plainly instead of disappearing.
- **Export as NCAP action-plan evidence (PRANA-ready CSV):** on the intervention-tracking card — every dispatched intervention with its measured effect, mapped to the NCAP spending head the city reports against and stamped with the city's regulatory authority. VayuNetra feeds the official portal rather than competing with it.

6.2 Forecast

- Horizon tabs **+24h / +48h / +72h**.
- Chart: city-average forecast line, shaded **80% uncertainty band**, and the gray dashed **persistence baseline** (what "tomorrow = today" would predict — the honest bar to beat).
- **Measured skill note:** e.g. "measured skill @24h: +12% vs persistence · +24% vs seasonal-naive · 80% band covers 78%" — read from the benchmark artifact (`GET /metrics/benchmark`), never typed in.
- **Forecast validation card** (below the chart): the strict temporal-split benchmark — skill vs persistence and weekly seasonal-naive per horizon, winter-only and Very-Poor-hours slices where they exist, 80% interval coverage, and the **early-warning line**: how many clean→Very-Poor *onsets* the model flags 24–72 h ahead (persistence = 0 by construction). Delhi has a multi-season run (2025-26 winter, 208k test hours, monthly refit on the trailing 90 days like production); every city has a live 90-day run. Toggle **multi-season / live 90d**. Negative numbers stay visible. See `docs/BENCHMARKS.md` , `docs/EARLY_WARNING.md` . Below it, **How today's attribution was made:** cells per method (per-cell model · shrunk to the city model mean · signature priors), median out-of-sample R^2 , mean confidence and how many cells reported a gas marker in the last 24 h (`GET /metrics/attribution`) — the split moves with station coverage and is shown, not smoothed.
- **Real interventions, in hindsight** (step 3): the CAQM GRAP escalations of winter 2025-26 (and Diwali night), dated from government releases, replayed against the served forecast — *would we have warned?* shows $P(>120)$ 24/48/72 h before each order and the share of station cells past the alarm; *did the air change?* shows observed vs weather-expected PM2.5 inside each window with a bootstrap interval. Low-coverage rows are flagged, negatives kept, every order links to its source. `docs/OUTCOMES.md` , `GET /metrics/interventions` .

- **Calibrated probability chips** in every Cell Story forecast tile: "**63% Very Poor+**" or "**91% Severe**" — split-conformal exceedance probabilities on held-out residuals, not thresholds on a point.
- **Spike alerts** when cells are forecast $\geq 90 \mu\text{g}/\text{m}^3$; scrollable per-cell list (worst first, interval + value).
- **City Statistics card: Who is in the forecast?** (expected people in Very Poor / Severe air at +24/48/72 h = Σ cell population $\times$ calibrated $P(> \text{band})$; GPW gridded population where sampled, cited city population otherwise; city-scaled with the assumption printed; person-hours over the outlook; exposure, not mortality — docs/HEALTH_IMPACT.md), then **City — past air** (daily PM2.5 for the whole city, 30 d / 90 d / 1 y with verdict and spike-day markers — Delhi's 1-year view shows the real winter smog season), the last-48h hourly PM2.5 area chart (real station readings), the live **source-mix donut** (mean attribution across live cells), and the "**Who breathes what**" stacked AQI-band bar (share of ~ 1 km cells in each CPCB band).

6.3 Advisories (the citizen loop)

Four numbered cards: **1 Citizen Advisory** · **2 Send it** · **3 Cleanest air right now** (with corridor screening) · **4 Citizen reports**. - **Language dropdown:** the city's showcase language first (Hindi in Delhi, Marathi in Mumbai/Pune, Kannada in Bengaluru), then English — full native-script text, generated by deterministic templates (**deliberately LLM-free:** templates cannot hallucinate medical advice; the health facts are locked). - **Channel tabs** show how the *same* advisory renders on each channel: - **App** — advisory cards per ward with risk-tier badges. - **Telegram** — a mock chat + QR code; the real bot (@aqivayu_bot) is live and two-way: /start → pick a city → auto-receive advisories. - **IVR call** — the spoken script, plus call-in: the line answers with a keypad menu of all ten cities (1 Delhi · 2 Bengaluru · 3 Mumbai · 4–9, 0 for the rest) and reads that city's latest advisory — **in Hindi (Polly Kajal, hi-IN) for Hindi-first cities**, English (Raveena) elsewhere; Polly has no Tamil/Telugu/Bengali/Gujarati/Kannada/Marathi voice yet, so those cities' calls read the English advisory (stated, not hidden). - **Big screen** — high-contrast public-display rendering. - "**Cleanest air right now**": the 4 lowest-PM2.5 ~ 1 km zones from the dense model field, each with a **Directions** link to Google Maps — explicitly labelled "a modeled guide, not a measurement". - "**Broadcast latest alert (Telegram + IVR)**": sends a *real* Telegram message and places a *real* phone call — with an are-you-sure confirmation step, and rate-limited server-side. - **Corridor exposure screening:** pick a clean-air zone as a destination and see the straight-corridor PM2.5 exposure from the city centre over the dense field vs the city mean — a planning guide, explicitly not turn-by-turn navigation. - 📸 **Report a pollution source (citizen complaint loop):** photo + category + location (GPS, or city centre as fallback) → the report appears in a public list with a **72-hour SLA clock**. An officer marking it *verified* registers the location as a candidate emission source so the next enforcement run scores it — the loop closes from the citizen side. Statuses: received → verified → actioned → resolved / rejected; SLA breaches are flagged in red.

6.4 Cities

- Grouped bar chart: each city's PM2.5 now vs +24h, and a **Swachh Vayu-style clean-air ranking** (cleanest first).
- Per-city cards: trend badge (deteriorating/stable), average, +24h, dominant source, signature match, health-burden line ($\sim$ deaths/yr, ₹/yr, "highest burden" flag), first playbook recommendation, and enforcement-compliance counts (approved/dispatched/dismissed).
- Powered by the Multi-City agent — cross-city comparative intelligence, not three copies of the same page.

6.5 Simulator (what-if + optimizer)

Three numbered cards: **1 Choose an intervention** · **2 Result** (empty until you run — it says so) · **3 Best bundle for a budget**. - Pick an **intervention** (waste-burn ban · halt construction dust · odd-even traffic · industrial shutdown · GRAP Stage III package) and a horizon → **Run simulation**. - Returns ΔAQI (average and best cell), cells affected, confidence, and the cited impact cards: **people protected** (real GPW population), **health cost avoided (₹)**, **deaths averted**, **CO₂e co-benefit** — every figure carries its citation (WHO HRAPIE CRFs, India SRS, Andreae 2019 ...); intervention magnitudes come from literature (Delhi odd-even $\approx 4\text{--}7\%$, CAQM GRAP schedules), not invented sliders. - **Honest-zero:** if the chosen source contributes $\sim 0\%$ today, the app says so ("Near-zero effect — honest result") instead of inventing

an impact. - **Optimizer:** set an inspector-hour budget (5–60h slider) → **Rank packages** → ranked intervention bundles by impact per inspector-hour.

6.6 Impact (the funding case)

Three numbered cards: **1 City ROI — the funding case** · **2 Where the funds should go** · **3 Fairness audit**. - **City ROI:** attributable deaths/yr, annual health burden (₹), and what a 30% NCAP reduction would avert — with the burden-vs-avertable bar chart, the narrative paragraph, and full citations (Chen & Hoek 2020 CRF, UN WUP population, IQAir annual mean). The ₹ figure is labelled order-of-magnitude, VSL caveat shown. - **Where the funds should go — attribution-weighted:** live source shares mapped to NCAP spending heads (e.g. "42% of PM2.5 is traffic → prioritise vehicular emission control") — the per-city answer to CREA's finding that NCAP cities put 67% of funds into road dust because allocation wasn't attribution-led. - **Fairness audit:** measured on every live recommendation — enforcement priority correlates with *pollution contribution* (dominant driver, by design) and with *population exposed* (deliberate, disclosed). Key guarantee: **no socio-economic inputs exist anywhere in the pipeline or schema** — the scorer sees only contribution, exposure, actionability, confidence.

6.7 Pipeline (the agents, visibly)

- Timeline of the last multi-agent run: **Orchestrator** → **Attribution** → **Forecast** → **Spike gate** → **Enforcement** → **Advisory** (+ Multi-City), each with cumulative time and per-node duration bars.
- The **spike gate** is a decision point: "no spike → straight to advisory" (green) or "spike → escalated to enforcement" (amber). On clean air, Enforcement shows as *"skipped — air is clean, nothing to enforce"* — the graph doesn't pretend to work.
- **"Run agents live"** triggers a real end-to-end run (POST /agent/query) and updates the trace + latency in front of you. Typical: **0.6–9.7 s**.

7. Under the hood (what to say when judges ask "how")

7.1 The six agents (LangGraph)

Agent	Does
A0 Orchestrator	Reads latest signals, finds spiking cells, routes the graph, stamps per-node latency traces
A1 Attribution	GBM + SHAP hybrid source attribution per cell; abstains to signature priors below the R ² gate
A2 Forecast	24/48/72h per-cell PM2.5 with CQR-calibrated intervals + persistence baseline
Spike gate	Branches: enforcement only when cells spike (PM2.5 > 120 µg/m ³ or focus cells exist)
A3 Enforcement	Priority = contribution × exposure × actionability × confidence; jurisdiction-aware RAG citations (GRAP/CAQM only in Delhi-NCR; CPCB/NCAP instruments elsewhere, issuing authority from city config); writes the worklist
A4 Advisory	Deterministic multilingual advisories, vulnerability-adjusted per ward/audience
A5 Multi-City	Cross-city trends, dominant sources, playbooks, health burden

Supporting models: **E1** satellite CV source detection · **E2** dense 1-km field (40 stations → ~900 cells/city) · **E3** counterfactual what-if · **E5** optimizer · **E7** health ₹/lives/CO₂e · GRAP forecast-trigger · multimodal RAG (text + Sentinel-2 patches in `kb_chunks`).

7.2 Data sources (all free/open)

CPCB CAAQMS (via data.gov.in / OpenAQ) · Open-Meteo + ERA5 weather & boundary layer · Sentinel-5P NO₂, Sentinel-2, NASA FIRMS fire (Earth Engine) · OpenStreetMap sources/facilities/roads (Overpass) · GPW v4.11 population (CIESIN/SEDAC) · GTFS + road network mobility proxy. Ingest runs on GitHub Actions crons (hourly + daily).

7.3 API (FastAPI, Render)

37 HTTP routes + 1 WebSocket, one envelope {success, data, error, meta} . The ones worth knowing:

Area	Endpoints
Feeds	<code>/cities</code> <code>/aqi/current</code> <code>/history</code> <code>/history/trend</code> (daily, per city or cell, with verdict + spike days) <code>/history/cells</code> (hourly per cell — the map replay) <code>/sources/{id}/patch</code> <code>/attribution</code> <code>/forecast</code> <code>/coverage</code> <code>/plume</code> <code>/static-layers</code> <code>/mobility</code>
Officer	<code>/enforcement</code> <code>/enforcement/{id}/dossier</code> <code>/enforcement/{id}/notice.pdf</code> POST <code>/enforcement/{id}/status</code> <code>/interventions</code> <code>/interventions/export</code> (PRANA-ready NCAP evidence CSV)
Citizen	<code>/advisory</code> POST <code>/advisory/broadcast</code> <code>/clean-zones</code> <code>/alerts/compound</code> POST <code>/telegram/webhook</code> <code>/ivr/inbound</code> <code>/ivr/advisory</code> POST <code>/report</code> <code>/reports</code> POST <code>/report/{id}/status</code> (complaint loop)
Analysis	<code>/comparison</code> <code>/roi</code> POST <code>/simulate</code> POST <code>/optimize</code> <code>/latency</code> <code>/traces</code>
Agents/Admin	POST <code>/agent/query</code> (runs the full graph) · POST <code>/admin/cities</code> (new city, key-gated) · WS <code>/live</code>

Auth: public reads via anon JWT under Postgres **RLS**; every write path is service-role, server-side only; admin behind X-Admin-Key ; Twilio TwiML endpoints are auth-less by necessity (Twilio can't send a bearer) and expose only public advisory text.

7.4 Database (Supabase Postgres + PostGIS + pgvector)

Core tables: cities, measurements, attribution, forecasts, emission_sources, enforcement_recgs, advisories, kb_chunks, action_traces + coverage_field, vulnerability, advisory_subscribers, intervention_tracking, profiles . 13 migrations in supabase/migrations/ (incl. citizen_reports).

7.5 Honesty, by construction (our differentiator — memorise these)

- Attribution **abstains** (R² gate + coverage mask) instead of guessing.
- Forecast intervals were audited, found under-covered, and **fixed with CQR** — the failure is documented, not hidden. Every forecast now also carries a **calibrated P(>120) / P(>250)**.
- The forecast is **benchmarked the way a reviewer would**: strict temporal split, production-faithful rolling refit, persistence + seasonal-naive + climatology baselines, one shared support mask, regime slices, onset recall, Brier skill, meteorology ablation — and the numbers (including the Severe-tail under-prediction and the -3.7% winter 24 h) are served by the API and printed in the console (docs/BENCHMARKS.md).
- Attribution is **compared with published apportionment** (TERI-ARAI 2018 for Delhi; Guttikunda 2019 and CSTEP 2022 for Bengaluru) and the disagreements (kerbside traffic over-weighted in monsoon Delhi; combustion point sources over-read and small waste fires under-read in Bengaluru) are stated (docs/ATTRIBUTION_VALIDATION.md).
- Every notice ends with a **PROVENANCE** section: figures are read from structured model output by deterministic code; regulatory text is retrieved verbatim; no number is produced by a language model. LLM fluency polish (optional, off by default) is gated by a facts check **and a script check** — a Hindi SMS with a stray CJK glyph is rejected, never sent.

- Positioning vs SAFAR / DSS / PRANA is written down with the CEEW audit numbers, including what we do **not** claim (docs/POSITIONING.md).
- A deep-learning model (TFT) was trained on GPU and **rejected** because LightGBM won held-out skill in every launch city.
- Health text is **LLM-free by design**; impact figures return **null** rather than use invented constants; <2% contributors never enter the worklist; notices are drafts, never auto-sent; demo fixtures are labelled as fixtures; nothing fabricated is ever written to the production DB.

7.6 Ops

make dev (run all) · make test (196 backend tests) · make benchmark (recompute live forecast benchmarks + fixtures) · web/scripts/qa/*.mjs (Playwright QA: section screenshots at projector/FHD/mobile, 10-city health sweep, deck + landing captures) · cd web && npx playwright test (7 smoke e2e; VN_LIVE=1 adds the 9-flow live officer journey) · make prewarm (GO/NO-GO) · make migrate / db-status · GitHub Actions: hourly ingest, daily model/registry/advisory refresh, keep-alive pings. New city = one YAML in core/config/cities/ (bbox, languages, stations) or one POST /admin/cities .

8. Troubleshooting & FAQ

Symptom	Meaning / fix
Amber "backend waking up" banner	Render free-tier cold start. Click retry or wait — clears on first successful call. Run make prewarm before demos.
Map loads, panels empty	Wrong city selected for the data you expect, or backend still waking. Switch city once, or check /cities responds.
"No per-cell forecast (see city panel)" in Cell Story	That cell has no fresh per-cell forecast rows — the city-level Forecast panel still works.
"No active recommendations"	Air is clean or filters exclude everything — click "Clear filters".
Advisory empty in a language	That city doesn't publish that language (languages come from city config, e.g. Kannada only in Bengaluru).
Notice PDF takes a few seconds	Live dossier assembly (RAG + satellite patch) takes 5–12 s — expected.
Tour won't reappear	It's once-only by design — replay via the ? button in the top bar.
Simulator says "Near-zero effect"	Honest result: the chosen source barely contributes today. Pick the dominant source instead.
Is what I'm seeing real or demo?	DEMO_MODE=false (production) = live Supabase data. DEMO_MODE=true = bundled fixtures (identical UX, labelled in /health).

Glossary. H3 — Uber's hexagonal grid; res-8 ≈ 1 km² cells; every table keys on h3_cell . **SHAP** — per-feature contribution of the model's prediction, in µg/m³. **CQR** — conformalized quantile regression; makes the 80% band actually cover ~80%. **GRAP** — Delhi-NCR's statutory Graded Response Action Plan (Stages I–IV). **RAG** — retrieval-augmented generation over the regulation corpus (citations, not free-text generation). **RLS** — Postgres row-level security. **Persistence** — the "tomorrow = today" baseline every forecast must beat. **GPW** — Gridded Population of the World v4.11 (NASA/CIESIN).

9. Repo map (where things live)

Path	What
<code>api/main.py</code>	All FastAPI routes
<code>agents/</code>	The 6 LangGraph agents + notice PDF writer
<code>ml/</code> , <code>core/</code>	Models, H3 utils, impact & intervention math, city configs
<code>connectors/</code>	CPCB, OpenAQ, Earth Engine, Open-Meteo, OSM, population, vulnerability, traffic, permits
<code>scripts/</code>	prewarm, refresh_advisories, bootstrap, freight corridors, LLM polish, seeds
<code>web/src/</code>	React console + landing (<code>App.tsx</code> shell, panels per section)
<code>demo/fixtures/</code>	The 17 JSON fixtures behind DEMO_MODE
<code>supabase/migrations/</code>	Schema, RLS, seeds (12 migrations)
<code>docs/</code>	This guide, pitch deck (<code>VayuNetra_Pitch.pptx</code>), demo script, architecture SVGs, methodology, API contract
<code>eval/evaluate.ipynb</code>	The full validation trail §§1–10, including the documented failures